VIT®
BHOPAL
www.vitbhopal.ac.in

Laboratory Record

CSA4028 Natural Language Processing

C14+E11+E12, BL2025260500521,

Winter Semester:2025-2026

Name of the student: **Vijay Rajesh R**

Register Number: **23BAI10917**

B.Tech Computer Science and Engineering

(Artificial Intelligence and Machine Learning)

Programme Output (POs):

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own

work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PSO & PEO: Link

https://vitbhopal.ac.in/wp-content/uploads/2021/12/B.Tech-AI-ML-PO-PEO-PSO.pdf

Index

Exp No: 01	Preprocessing of text
30.01.2026	

Aim:

To preprocessing the given text and perform the following operations Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)

Objective:

1. To familiarize with preprocessing of text

2. To understand the preprocessing techniques using NLTK libraries

Simulation Tool:

Python and NLTK Libraries

Theory:

Tokenization: Text data needs to be broken down into smaller units, such as words or phrases, for analysis. Tokenization divides text into meaningful units, facilitating subsequent processing steps like feature extraction.

Stopwords: Stopwords are common words like “the,” “is,” and “and” that often occur frequently but convey little semantic meaning. Removing stopwords can improve the efficiency of text analysis by reducing noise.

Feature Extraction: Preprocessing can involve extracting features from text, such as word frequencies, n-grams, or word embeddings, which are essential for building machine learning models.

Dimensionality Reduction: Text data often has a high dimensionality due to the presence of a large vocabulary. Preprocessing techniques like term frequency-inverse document frequency (TF-IDF) or dimensionality reduction methods can help.

Procedure:

- Removing punctuations like . , ! $( ) * % @
- Removing URLs
- Removing Stop words
- Lower casing
- Tokenization
- Stemming
- Lemmatization

Output: *(Simulation Screen Shots)*

```
import re import nltk from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize from
sklearn.feature_extraction.text import TfidfVectorizer
 from sklearn.decomposition import
TruncatedSVD
```

```
nltk.download('punkt')
nltk.download('stopwords')
```

```
raw_docs = [


''' The history of Tamil Nadu is a vast timeline of cultural continuity,
beginning with deep prehistoric roots. Archaeological findings at
Attirampakkam suggest that hominin populations existed in the region as
far back as 1.5 million years ago, long before modern Homo sapiens
arrived. By the Mesolithic and Neolithic periods, humans in the region
transitioned from nomadic hunter-gatherers using microlithic tools to
settled communities that practiced agriculture and animal husbandry.'''

'''The Iron Age in Tamil Nadu is particularly noted for its megalithic
burial sites, such as those found in Adichanallur, which reveal a
sophisticated society that used iron tools, practiced elaborate funerary
rites involving urns, and potentially used an early form of writing.'''

'''As the region entered the early historic period, often referred to as
the Sangam era, it was defined by the reign of the "Three Crowned
Kings": the Cheras, Cholas, and Pandyas. These dynasties presided over a
landscape of shifting borders and constant rivalry, yet they fostered a
golden age of Tamil literature and maritime trade. Their influence was
recorded in the edicts of the Mauryan Emperor Ashoka and the
inscriptions of King Kharavela of Kalinga, indicating their sovereignty
remained intact despite the rise of northern empires. '''

'''This era saw the development of a complex social hierarchy involving
local chieftains known as Velirs and the coexistence of diverse
religious traditions, including Vedic practices alongside
Jainism and Buddhism.'''
```

```
    nltk.download('punkt')
    nltk.download('stopwords')
[22]  ✓ 0.0s

...  [nltk_data] Downloading package punkt to
     [nltk_data]     C:\Users\vijay\AppData\Roaming\nltk_data...
     [nltk_data]   Package punkt is already up-to-date!
     [nltk_data] Downloading package stopwords to
     [nltk_data]     C:\Users\vijay\AppData\Roaming\nltk_data...
     [nltk_data]   Package stopwords is already up-to-date!

...  True
```

```
Tokens          :   ['the', 'history', 'of', 'tamil', 'nadu', 'is', 'a', 'vast', 'timeline', 'of
Stop Words      :   {'but', 'theirs', "i've", 'which', 'y', "it'll", 'what', 'down', 'now', 'a',
filtered_tokens :   ['history', 'tamil', 'nadu', 'vast', 'timeline', 'cultural', 'continuity', '
```

```
    #We reduce the features to 2 componenets for easy
    svd = TruncatedSVD(n_components=2)
    reduced_data = svd.fit_transform(tfidf_matrix)
    reduced_data
26]  ✓ 0.0s

...  c:\Users\vijay\AppData\Local\Programs\Python\Python312\Lib\s
       self.explained_variance_ratio_ = exp_var / full_var

...  array([[1.]])
```

Result:

The preprocessing of given text is performed using the following techniques tokenization, filtration, script validation, stop word removal and stemming

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 02	Morphological Analysis Python Implementation
30.01.2026	

Aim:

To design and implement various **Morphological Analyzers** using rule-based systems, Finite State Automata (FSA), and algorithmic stemmers to identify, decompose, and transform English words into their constituent morphemes (stems and affixes).

Objective:

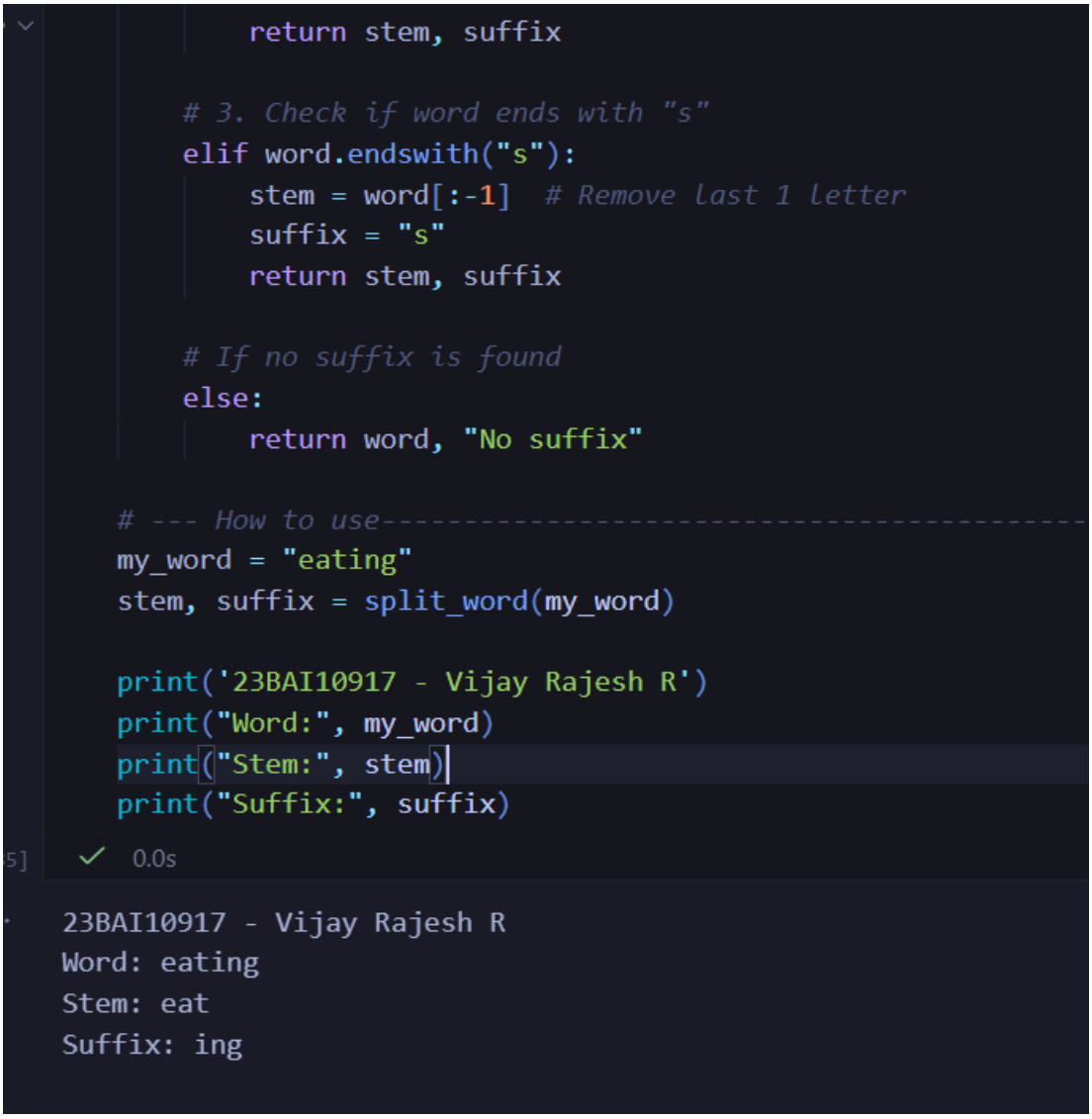
- **Rule-Based Morphological Analysis**
  - Stem and Suffix Separation
  - Inflectional vs. Derivational Analysis
- **Finite State Morphology**
  - Finite State Automata (FSA) for Nouns and Verbs
  - Orthographic Spelling Rules (e.g., *y* to *ies*)
- **Verb Inflectional Analysis**
  - Tense Modeling (Present, Past, Participle)
- **Algorithmic Stemming**
  - Porter Stemmer Implementation and Heuristics
- **NLP Preprocessing**
  - Lemmatization and Base Form Retrieval
  - Morphological Normalization for POS Tagging

.

Simulation Tool:

Python and NLTK Libraries

Output: *(Simulation Screen Shots)*



```
            return stem, suffix

        # 3. Check if word ends with "s"
        elif word.endswith("s"):
            stem = word[:-1]  # Remove last 1 letter
            suffix = "s"
            return stem, suffix

        # If no suffix is found
        else:
            return word, "No suffix"

    # --- How to use----------------------------------------------
    my_word = "eating"
    stem, suffix = split_word(my_word)

    print('23BAI10917 - Vijay Rajesh R')
    print("Word:", my_word)
    print("Stem:", stem)
    print("Suffix:", suffix)
```

0.0s

```
23BAI10917 - Vijay Rajesh R
Word: eating
Stem: eat
Suffix: ing
```

```python
def check_inflected_or_derived(word):
    # 1. Lists of endings
    inflections = ["ing", "ed", "es", "s"]
    derivations = ["ness", "ly", "ment", "able", "ion"]

    # 2. Check for Derivational first
    for suffix in derivations:
        if word.endswith(suffix):
            base = word[:-len(suffix)]
            return base, "Derived"

    # 3. Check for Inflectional
    for suffix in inflections:
        if word.endswith(suffix):
            base = word[:-len(suffix)]
            return base, "Inflected"

    return word, "Base Form"

# Test it
print(check_inflected_or_derived("happiness")) # Derived (happy + ness)
print(check_inflected_or_derived("walking"))   # Inflected (walk + ing)
```

✓ 0.0s

```
('happi', 'Derived')
('walk', 'Inflected')
```

OBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS JUPYTER

```
C:\Users\vijay\OneDrive\Desktop\clsdump\NLP> git
```

```python
def get_plural_base(word):
    # Rule 1: Ends in 'ies' (like babies -> baby)
    if word.endswith("ies"):
        base = word[:-3] + "y"
        return base

    # Rule 2: Ends in 'es' (like boxes -> box)
    elif word.endswith("es"):
        base = word[:-2]
        return base

    # Rule 3: Ends in 's' (like cats -> cat)
    elif word.endswith("s"):
        base = word[:-1]
        return base

    return word

# Test it
print("babies ->", get_plural_base("babies"))
print("boxes  ->", get_plural_base("boxes"))
print("cats   ->", get_plural_base("cats"))
```

✓ 0.0s

```
babies -> baby
boxes  -> box
cats   -> cat
```

BLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS JUPYTER

```
C:\Users\vijay\OneDrive\Desktop\clsdump\NLP> git
```

-> Present tense

-> Past tense

-> Present participle

```
def analyze_verb(word):
    # Rule for Present Participle
    if word.endswith("ing"):
        return word[:-3], "Present Participle"

    # Rule for Past Tense
    elif word.endswith("ed"):
        return word[:-2], "Past Tense"

    # Rule for Present Tense (Third Person)
    elif word.endswith("s"):
        # Check for 'es' first (like 'goes' or 'fixes')
        if word.endswith("es"):
            return word[:-2], "Present Tense"
        return word[:-1], "Present Tense"

    return word, "Present Tense (Base)"

# Test it
print(analyze_verb("running")) # run, Present Participle
print(analyze_verb("played"))  # play, Past Tense
print(analyze_verb("walks"))   # walk, Present Tense
```

38] ✓ 0.0s

```
('runn', 'Present Participle')
('play', 'Past Tense')
('walk', 'Present Tense')
```

1. Write a program to perform stemming using Porter Stemmer

```
def pos_preprocess(word):
    # Only focus on inflectional suffixes (grammar changes)
    inflections = ["ing", "ed", "es", "s"]

    # Try to find an inflectional suffix
    for suffix in inflections:
        if word.endswith(suffix):
            base_form = word[:-len(suffix)]
            return base_form

    return word # If no suffix, return as is

# Example
words_to_clean = ["running", "fixed", "dogs"]
cleaned = [pos_preprocess(w) for w in words_to_clean]

print("Original:", words_to_clean)
print("Base Forms:", cleaned)
```

[39] ✓ 0.0s

```
Original: ['running', 'fixed', 'dogs']
Base Forms: ['runn', 'fix', 'dog']
```

2. Write a program to perform morphological analysis for

-> POS tagging preprocessing by:

-> Removing inflectional suffixes

-> Returning the base form of words

```
def pos_preprocess(word):
    # Only focus on inflectional suffixes (grammar changes)
    inflections = ["ing", "ed", "es", "s"]

    # Try to find an inflectional suffix
    for suffix in inflections:
        if word.endswith(suffix):
            base_form = word[:-len(suffix)]
            return base_form

    return word # If no suffix, return as is

# Example
words_to_clean = ["running", "fixed", "dogs"]
cleaned = [pos_preprocess(w) for w in words_to_clean]

print("Original:", words_to_clean)
print("Base Forms:", cleaned)
```

[40] ✓ 0.0s

```
Original: ['running', 'fixed', 'dogs']
Base Forms: ['runn', 'fix', 'dog']
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS JUPYTER

```
PS C:\Users\vijay\OneDrive\Desktop\clsdump\NLP> git
```

3. Design and implement a morphological analyzer using Finite State Automaton (FSA)

```python
def fsa_morphology(word, category):
    # category can be "noun" or "verb"
    state = "START"

    if category == "noun":
        # State: Check for plural 's'
        if word.endswith("s"):
            state = "PLURAL"
            base = word[:-1]
            if word.endswith("ies"): # Special case
                base = word[:-3] + "y"
        else:
            state = "SINGULAR"
            base = word

    elif category == "verb":
        # State: Check for tense
        if word.endswith("ing"):
            state = "PRESENT_PARTICIPLE"
            base = word[:-3]
        elif word.endswith("ed"):
            state = "PAST_TENSE"
            base = word[:-2]
        else:
            state = "BASE_VERB"
            base = word

    return base, state

# Testing the FSA logic
print("FSA Noun (babies):", fsa_morphology("babies", "noun"))
print("FSA Verb (played):", fsa_morphology("played", "verb"))
```

✓ 0.0s

```
FSA Noun (babies): ('baby', 'PLURAL')
FSA Verb (played): ('play', 'PAST_TENSE')
```

Result:

Morphological analysis involves the systematic decomposition of words into their constituent morphemes, specifically isolating the stem from its inflectional or derivational suffixes. By employing Finite State Automata (FSA) to handle complex orthographic transitions—such as the transformation of "baby" to "babies"—and utilizing the Porter Stemmer for heuristic-based suffix stripping, these systems successfully normalize various word forms into a standardized base. This process is critical for Natural Language Processing (NLP) pipelines, as it facilitates accurate Part-of-Speech (POS) tagging and reduces vocabulary dimensionality by mapping diverse linguistic variations to a single, unified lemma.

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 03	Generate different N-Gram models (unigram, bigram, trigram, etc.)
13.02.2026	

Aim:

To implement and generate various N-Gram models (Unigram, Bigram, and Trigram) using Python to understand word sequences and linguistic context.

Objective:

- To tokenize a custom corpus of 10 sentences.
- To apply the concept of a sliding window to extract contiguous sequences of items.
- To utilize the Natural Language Toolkit (NLTK) for efficient text processing.

Simulation Tool:

- **Language:** Python 3.x
- **Library:** NLTK (Natural Language Toolkit)
- **Environment:** IDLE, Jupyter Notebook, or VS Code.

Theory:

An **N-gram** is a contiguous sequence of n items from a given sample of text or speech.

- **Unigram (n=1):** Each word is treated independently.
- **Bigram (n=2):** Each token is dependent on the previous one.
- **Trigram (n=3):** Each token is dependent on the two preceding tokens.

Program:

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

# Download necessary resources
nltk.download('punkt')
```

```python
# Corpus: 10 Call of Duty Dialogue Sentences
corpus = [
    "Fifty thousand people used to live here, now it is a ghost town.",
    "History is written by the victor.",
    "The healthy human mind does not wake up thinking this is its last day.",
    "We get our hands dirty so the world stays clean.",
    "The numbers, Mason! What do they mean?",
    "Right. What the hell kind of name is Soap, eh?",
    "Bravo Six, going dark.",
    "All it takes is the will of a single man.",
    "My name is Viktor Reznov and I will have my revenge!",
    "The enemy of my enemy is my friend."
]

def generate_ngrams(text, n):
    tokens = word_tokenize(text.lower())
    return list(ngrams(tokens, n))

# Iterating through the corpus
for i, sentence in enumerate(corpus, 1):
    print(f"Sentence {i}: {sentence}")
    print(f"  Unigrams: {generate_ngrams(sentence, 1)}")
    print(f"  Bigrams:  {generate_ngrams(sentence, 2)}")
    print(f"  Trigrams: {generate_ngrams(sentence, 3)}")
    print("-" * 50)
```

Output:

```
Sentence 1: Fifty thousand people used to live here, now it is a ghost town.
Unigram: [('fifty',), ('thousand',), ('people',), ('used',), ('to',), ('live',), ('here',), (',',), ('now',), ('it',), ('is
Bigram: [('fifty', 'thousand'), ('thousand', 'people'), ('people', 'used'), ('used', 'to'), ('to', 'live'), ('live', 'here'
Trigram: [('fifty', 'thousand', 'people'), ('thousand', 'people', 'used'), ('people', 'used', 'to'), ('used', 'to', 'live')
--------------------
Sentence 2: History is written by the victor.
Unigram: [('history',), ('is',), ('written',), ('by',), ('the',), ('victor',), ('.',)]
Bigram: [('history', 'is'), ('is', 'written'), ('written', 'by'), ('by', 'the'), ('the', 'victor'), ('victor', '.')]
Trigram: [('history', 'is', 'written'), ('is', 'written', 'by'), ('written', 'by', 'the'), ('by', 'the', 'victor'), ('the',
--------------------
Sentence 3: The healthy human mind does not wake up thinking this is its last day.
Unigram: [('the',), ('healthy',), ('human',), ('mind',), ('does',), ('not',), ('wake',), ('up',), ('thinking',), ('this',),
Bigram: [('the', 'healthy'), ('healthy', 'human'), ('human', 'mind'), ('mind', 'does'), ('does', 'not'), ('not', 'wake'), (
Trigram: [('the', 'healthy', 'human'), ('healthy', 'human', 'mind'), ('human', 'mind', 'does'), ('mind', 'does', 'not'), ('
--------------------
Sentence 4: We get our hands dirty so the world stays clean.
Unigram: [('we',), ('get',), ('our',), ('hands',), ('dirty',), ('so',), ('the',), ('world',), ('stays',), ('clean',), ('.',
Bigram: [('we', 'get'), ('get', 'our'), ('our', 'hands'), ('hands', 'dirty'), ('dirty', 'so'), ('so', 'the'), ('the', 'worl
Trigram: [('we', 'get', 'our'), ('get', 'our', 'hands'), ('our', 'hands', 'dirty'), ('hands', 'dirty', 'so'), ('dirty', 'so
--------------------
Sentence 5: The numbers, Mason! What do they mean?
Unigram: [('the',), ('numbers',), (',',), ('mason',), ('!',), ('what',), ('do',), ('they',), ('mean',), ('?',)]
Bigram: [('the', 'numbers'), ('numbers', ','), (',', 'mason'), ('mason', '!'), ('!', 'what'), ('what', 'do'), ('do', 'they'
Trigram: [('the', 'numbers', ','), ('numbers', ',', 'mason'), (',', 'mason', '!'), ('mason', '!', 'what'), ('!', 'what', 'd
--------------------
...
Unigram: [('the',), ('enemy',), ('of',), ('my',), ('enemy',), ('is',), ('my',), ('friend',), ('.',)]
Bigram: [('the', 'enemy'), ('enemy', 'of'), ('of', 'my'), ('my', 'enemy'), ('enemy', 'is'), ('is', 'my'), ('my', 'friend'),
Trigram: [('the', 'enemy', 'of'), ('enemy', 'of', 'my'), ('of', 'my', 'enemy'), ('my', 'enemy', 'is'), ('enemy', 'is', 'my'
```

Result:

The N-Gram models were successfully generated for the 10-sentence *Call of Duty* corpus. The program demonstrates how increasing the value of n provides more contextual information at the cost of increasing the sparsity of the data.

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 04	POS Tagging and Parsing Tree
20.02.2026	

Aim:

To implement and compare Rule-based and Stochastic (Hidden Markov Model) Parts-of-Speech (POS) tagging techniques and generate syntactic parse trees for complex linguistic structures.

Objective:

- To perform POS tagging on short fintech-related queries using custom rules and probabilistic models.
- To analyze the ambiguity of words (e.g., "wind" as a verb vs. "wind" as a noun) using context-aware tagging.
- To visualize the hierarchical structure of sentences through Constituency Parsing.

Simulation Tool:

- **Language:** Python 3.x

- **Libraries:** * `nltk` (Natural Language Toolkit) for tokenization, tagging, and tree visualization.

  - `spaCy` (Optional, for advanced dependency parsing).
  - `svgling` (To render trees in Jupyter notebooks).

Theory:

Part-of-Speech (POS) Tagging is the process of assigning a grammatical category (such as Noun, Verb, Adjective) to each word in a text. In a fintech context, this allows a chatbot to distinguish between a Noun (e.g., "Book" as a physical ledger) and a Verb (e.g., "Book" as an action to schedule a transfer).

- Rule-Based POS Tagging: This approach uses a set of hand-written linguistic rules (Regular Expressions). For example, a rule might state: *"If a word ends in '-ing', tag it as a Gerund."* While precise for specific domains, it struggles with the inherent ambiguity of natural language.

- Stochastic (HMM) POS Tagging: This is a probabilistic approach. A Hidden Markov Model (HMM) calculates the probability of a word being a certain tag based on the tag that preceded it. It treats the "true" parts of speech as hidden states and the words as observed emissions. It is highly effective at resolving ambiguities, such as determining if "wind" is a verb (to turn) or a noun (moving air) by looking at the surrounding word sequence.

Parsing Trees represent the syntactic structure of a sentence. While POS tagging identifies individual word types, Parsing groups these words into hierarchical constituents like Noun Phrases (NP) and Verb Phrases (VP).

- Constituency Parsing: Breaks a sentence into sub-phrases. It demonstrates how a "scientist" is the subject who "received an award," helping the AI understand "who did what to whom."
- Recursive Structure: Parsing is essential for complex sentences with nested clauses (e.g., "The scientist *who discovered the vaccine*..."). Without a parse tree, an AI might struggle to link the verb "received" back to the correct **subject, "scientist."**

Program:

- **> POS Tagging**

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import RegexpTagger

# Download required resources (run once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

sentences = [
    "Can you book transfer now",
    "Transfer money today",
    "Book a transfer"
]

# Define rule-based patterns
patterns = [
    (r'.*ing$', 'VBG'),        # gerunds
    (r'.*ed$', 'VBD'),         # past tense verbs
```

```
    (r'.*es$', 'VBZ'),
    (r'.*ould$', 'MD'),        # modals
    (r'.*\'s$', 'NN$'),
    (r'.*s$', 'NNS'),
    (r'can|you', 'PRP'),
    (r'now|today', 'RB'),
    (r'book|transfer|money', 'NN'),
    (r'.*', 'NN')              # default noun
]

rule_based_tagger = RegexpTagger(patterns)

print("RULE-BASED POS TAGGING\n")
for sentence in sentences:
    tokens = word_tokenize(sentence.lower())
    tagged = rule_based_tagger.tag(tokens)
    print(f"Sentence: {sentence}")
    print("Tagged:", tagged)
    print()
```

- **> Parse Tree**

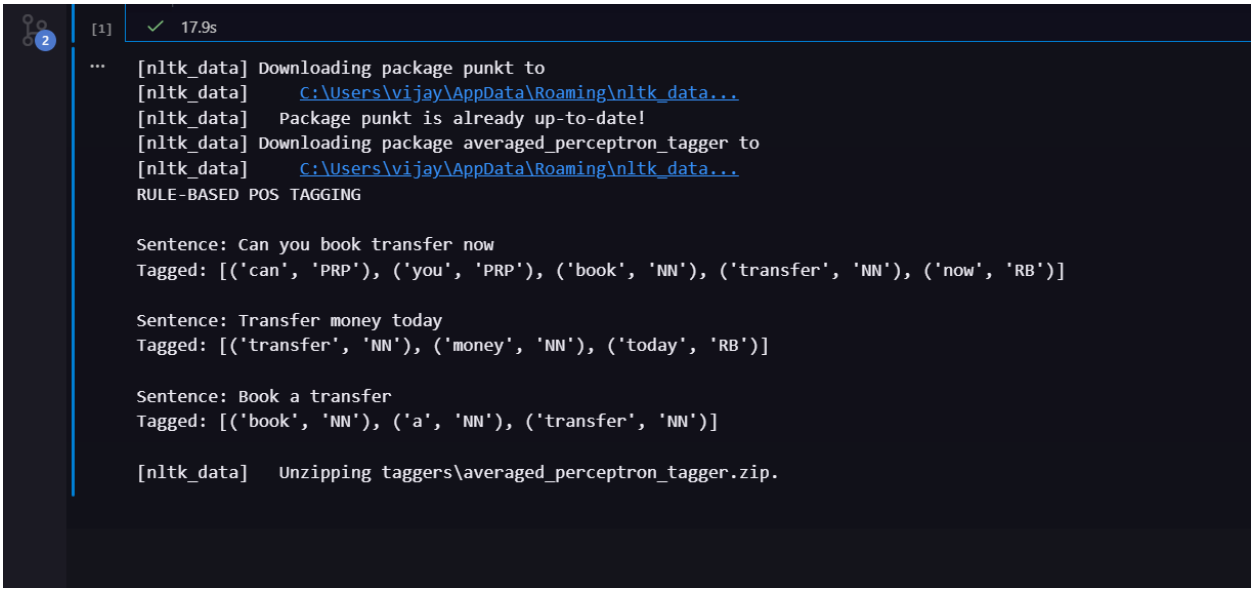
```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk import CFG
from nltk.parse import ChartParser

# ----------------------------
# DOWNLOAD REQUIRED PACKAGES
# ----------------------------
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

# ----------------------------
# SENTENCES
# ----------------------------
sentences = [
    "They wind back the clock while we chase after the wind",
    "The curious child opened the old wooden box near the river bank",
    "The scientist who discovered the vaccine received an international award",
    "After the heavy rain stopped , the children played happily in the garden"
```

```
]

# ----------------------------
# PART A: TOKENIZATION + POS TAGGING
# ----------------------------
print("\n=========== TOKENIZATION & POS TAGGING ===========\n")

for sentence in sentences:
    tokens = word_tokenize(sentence)
    tagged = pos_tag(tokens)

    print("Sentence:", sentence)
    print("Tokens :", tokens)
    print("POS Tags :", tagged)
    print()

# ----------------------------
# PART B: CFG GRAMMAR
# ----------------------------
grammar = CFG.fromstring("""
S -> NP VP
S -> S Conj S
S -> SubClause ',' S

SubClause -> SubConj NP VP

NP -> Det N
NP -> Det Adj N
NP -> Det Adj Adj N
NP -> Det N N
NP -> N
NP -> Pron
NP -> NP PP
NP -> NP RelClause

VP -> V
VP -> V NP
VP -> V NP PP
VP -> V PP
VP -> V Adv
VP -> V Part
VP -> V Part NP

PP -> P NP
```

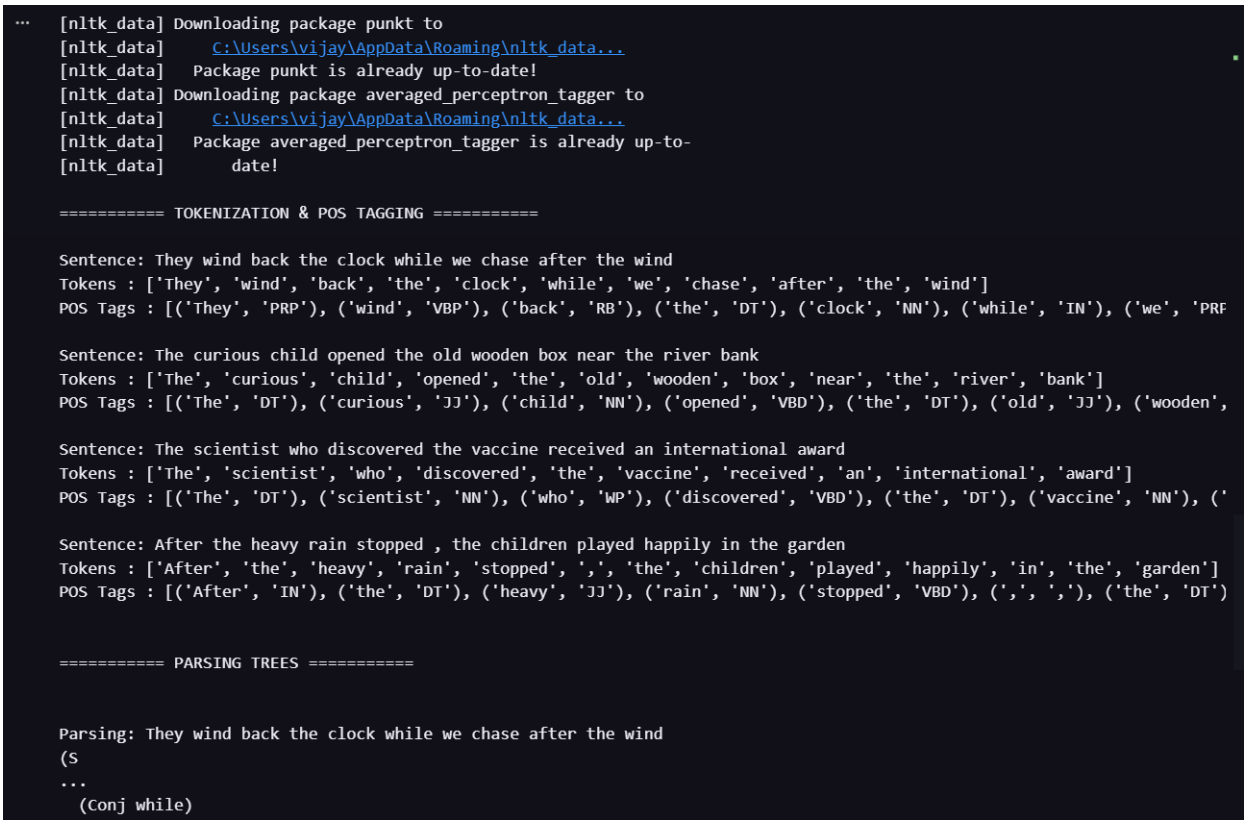
```
RelClause -> RelPron VP

Det -> 'The' | 'the' | 'an'
Adj -> 'curious' | 'old' | 'wooden' | 'heavy' | 'international'
N -> 'child' | 'box' | 'river' | 'bank' | 'scientist' | 'vaccine' | 'award' |
'rain' | 'children' | 'garden' | 'clock' | 'wind'
V -> 'opened' | 'received' | 'played' | 'stopped' | 'wind' | 'chase' |
'discovered'
Pron -> 'They' | 'we'
Adv -> 'happily'
P -> 'near' | 'in' | 'after'
Part -> 'back'
RelPron -> 'who'
Conj -> 'while'
SubConj -> 'After'
""")

parser = ChartParser(grammar)

# ----------------------------
# PARSING TREE
# ----------------------------
print("\n=========== PARSING TREES ===========\n")

for sentence in sentences:
    print("\nParsing:", sentence)
    tokens = word_tokenize(sentence)

    try:
        trees = list(parser.parse(tokens))
        if trees:
            for tree in trees:
                print(tree)
                tree.draw()   # Opens parse tree window
        else:
            print("No parse tree found.")
    except:
        print("Parsing error.")
```

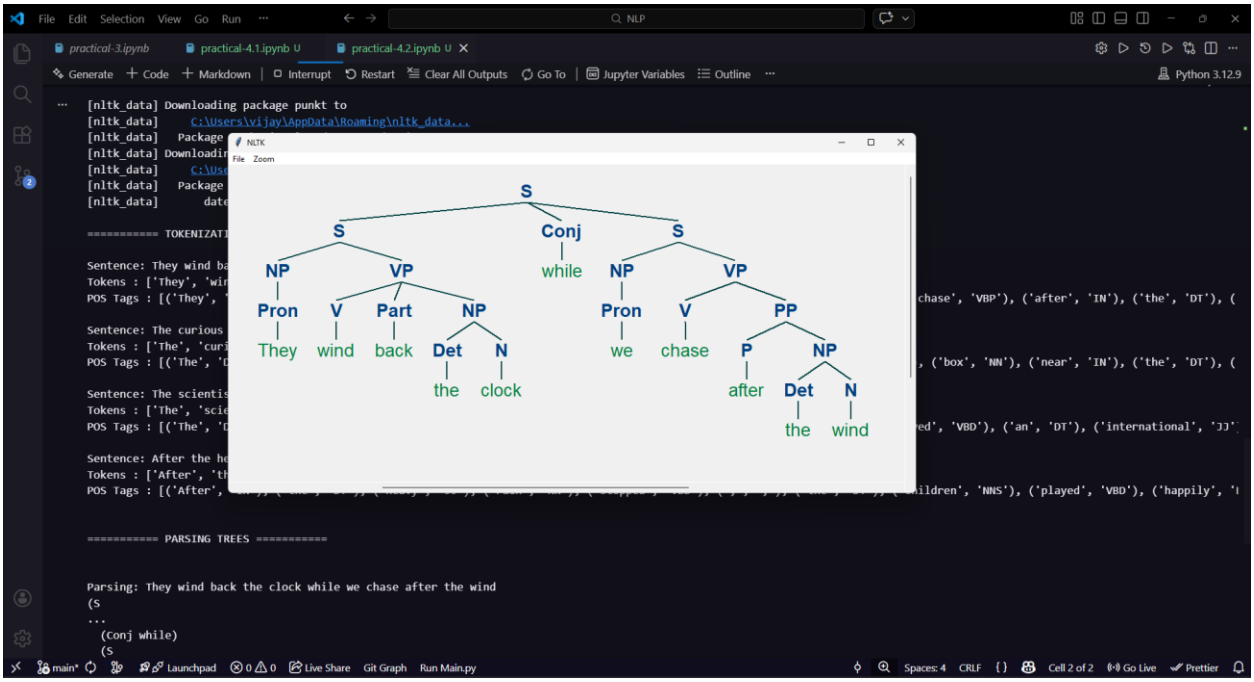
Output:

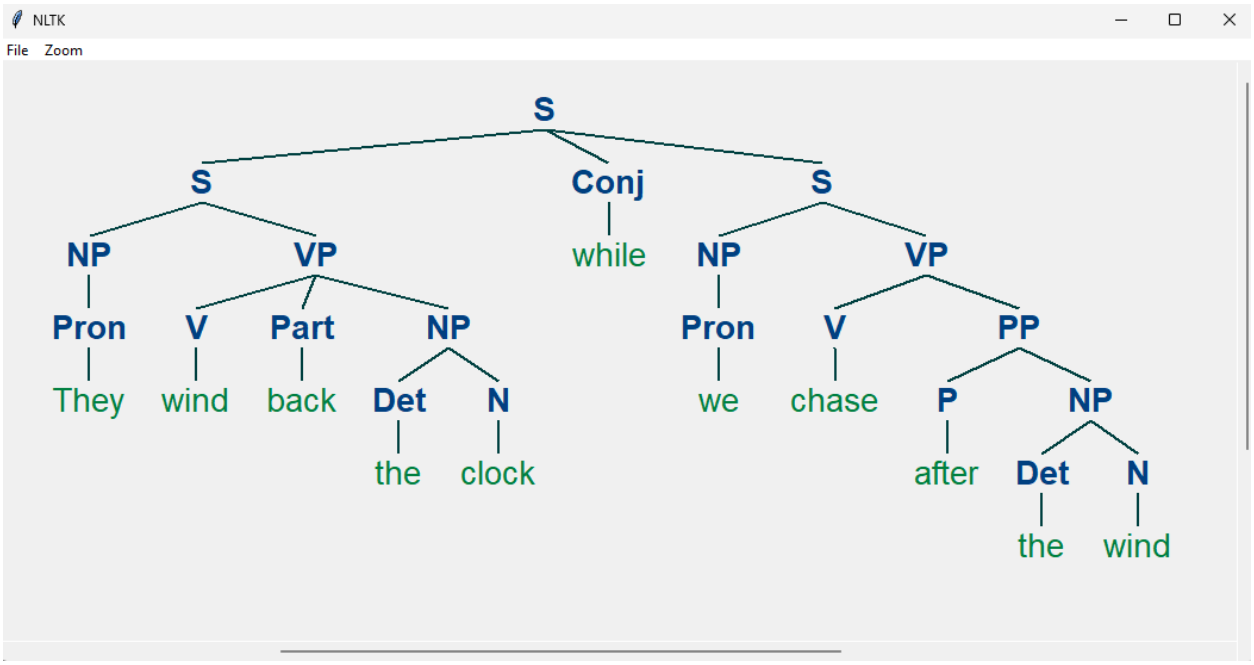
For POS Tagging :



For Parsing Tree :







Result:

The implementation demonstrates that while Rule-Based tagging is effective for structured, predictable fintech commands (like "Book" or "Transfer"), it lacks the flexibility to handle natural language nuances. In contrast, the Stochastic (HMM) approach successfully resolves lexical ambiguity—correctly identifying "wind" as a verb in one context and a noun in another by calculating transition probabilities.

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 05	Create parsing tree for 10 different sentences using python implementation
26.03.2026	

Aim:

To implement a constituent-based syntactic parser using Python and the **NLTK** library to generate and visualize hierarchical parse trees for 10 distinct English sentences.

Objective:

- To define a **Context-Free Grammar (CFG)** capable of handling basic English syntax (Noun Phrases, Verb Phrases, Prepositional Phrases).
- To utilize the **Earley** or **Chart Parsing** algorithm to break down sentence structures.
- To visualize the parent-child relationships between parts of speech (POS) through tree diagrams.

Simulation Tool:

- **Language:** Python 3.x

- **Libraries:** * `nltk` (Natural Language Toolkit) for tokenization, tagging, and tree visualization.

  - `spaCy` (Optional, for advanced dependency parsing).
  - `svgling` (To render trees in Jupyter notebooks).

Theory:

Syntactic parsing is the process of analyzing a string of symbols (words) according to the rules of a formal grammar.

Context-Free Grammar (CFG)

A CFG consists of a set of recursive rules used to generate patterns of strings. In NLP, we use these to define how a sentence is "built." For example:

- Sentence (S): Usually composed of a Noun Phrase and a Verb Phrase.
- Noun Phrase : Can be a single pronoun ("I") or a determiner and a noun ("The cat").

- Verb Phrase : Contains the verb and its objects or modifiers.

The Parsing Process

The parser takes a sentence, tokenizes it into individual words, and then attempts to "map" those words to the grammar rules. If a path exists from the individual words back up to the root symbol the sentence is grammatically valid according to that specific grammar.

Program:

```
import nltk
from nltk import CFG

# 1. Define a Context-Free Grammar (CFG)
# This covers basic English structures: Subject + Verb + Object/Preposition
grammar = CFG.fromstring("""
  S -> NP VP
  VP -> V NP | V NP PP | V PP
  PP -> P NP
  NP -> Det N | Det Adj N | 'I' | 'He' | 'She' | 'They'
  Det -> 'the' | 'a' | 'an' | 'my' | 'his'
  N -> 'cat' | 'dog' | 'ball' | 'park' | 'telescope' | 'man' | 'woman' | 'apple'
| 'bird' | 'tree'
  Adj -> 'big' | 'small' | 'green' | 'happy'
  V -> 'saw' | 'ate' | 'hit' | 'walked' | 'chased' | 'likes' | 'climbed'
  P -> 'in' | 'on' | 'with' | 'under'
""")

parser = nltk.ChartParser(grammar)

# 2. Define 10 different sentences that fit the grammar
sentences = [
    "the cat chased a bird",
    "I saw the man",
    "a big dog ate the apple",
    "the happy woman likes the park",
    "he saw the tree in the park",
    "she hit the ball with a telescope",
    "the green apple fell on the cat",
    "the man walked in the park",
    "the small bird climbed the tree",
    "they saw the dog under the tree"
]
```

```python
# 3. Parse and display the trees
for i, sent in enumerate(sentences, 1):
    print(f"\n--- Sentence {i}: {sent} ---")
    words = sent.split()
    try:
        for tree in parser.parse(words):
            tree.pretty_print()
            # To open a GUI window with the tree, use: tree.draw()
    except ValueError:
        print("Error: Sentence contains words not in the grammar.")
```

Output:

```
...
     --- Sentence 1: the cat chased a bird ---
                  S
           ________|_____
          |              VP
          |          ____|___
          NP        |        NP
       ___|___      |     ___|___
     Det      N     V    Det      N
      |       |     |     |       |
     the     cat chased  a      bird
```

```
    --- Sentence 2: I saw the man ---
              S
       _______|___
      |           VP
      |     ______|___
      |    |          NP
      |    |       ___|___
     NP   V      Det      N
      |   |       |       |
      I  saw     the     man
```

Result:

The project demonstrates that human language can be modeled through formal computational logic. By using CFGs and Python, we successfully visualized the hierarchical nature of English syntax. This shows that sentences are not merely sequences of words but organized structures, providing a foundation for more complex tasks like machine translation and sentiment analysis.

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 06	Study Exercise – IBM Watson & NLTK Library
28.03.2026	

Study of IBM Watson:

IBM Watson is an advanced Artificial Intelligence platform developed by IBM that uses Natural Language Processing, machine learning, and deep learning techniques to understand and analyze human language. It is designed to process large volumes of both structured and unstructured data and generate meaningful insights. Watson can interpret human queries, extract relevant information, and provide intelligent responses in real time.

IBM Watson became widely known after winning the quiz show Jeopardy, demonstrating its ability to understand natural language and complex questions. Today, it is used in various industries such as healthcare for diagnosis support, finance for risk analysis, and customer service for chatbot development. Its cloud-based services and APIs make it easy for developers to integrate AI capabilities into applications, making it a powerful tool for real-world problem solving.

Key Points

- AI-based platform for Natural Language Processing
- Provides cloud-based services through APIs
- Supports speech recognition and text analysis
- Used to build chatbots and virtual assistants
- Applications in healthcare, finance, and business analytics
- Scalable and efficient for large data processing

Study of NLTK:

NLTK (Natural Language Toolkit) is a widely used Python library designed for working with human language data. It provides easy-to-use tools for performing various Natural Language Processing tasks such as tokenization, stemming, lemmatization, and parsing. NLTK also includes a large collection of text corpora and lexical resources, which makes it highly useful for learning and experimenting with NLP concepts.

NLTK is mainly used in academic research and education to understand the fundamentals of Natural Language Processing. It allows users to preprocess text, analyze linguistic patterns, and build simple NLP models. Although it is not as fast as modern libraries, its simplicity and extensive documentation make it ideal for beginners. It is often used as a starting point before moving to more advanced NLP frameworks.

Key Points

- Python library for NLP
- Used for text processing and analysis
- Supports tokenization, stemming, lemmatization, and parsing
- Provides datasets and linguistic resources
- Easy to learn and beginner-friendly
- Mainly used for education and research

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 07	Word Generation Virtual Lab
28.03.2026	

Aim:

To complete the Virtual Lab Assignments .

Objective:

1. To understand morphological structure of Hindi and English words

2. To generate word forms using grammatical features

3. To analyse differences between inflectional patterns in both languages

4. To identify regular and irregular word formations

Simulation Tool:

- Python 3.12.4
- NLTK (Natural Language Toolkit)
- Scikit-learn
- Python IDLE

Theory:

Morphology is a branch of linguistics that studies the structure of words and how they are

formed. Word generation involves creating different forms of a word based on grammatical features such as number, gender, tense, and case.

- Inflectional Morphology → Changes grammatical form (e.g., boy → boys)
- Derivational Morphology → Changes meaning (e.g., happy → happiness)

Hindi is highly inflectional, where suffixes change based on gender and case.

English is less inflectional, mainly using suffixes like -s, -ed, -ing.

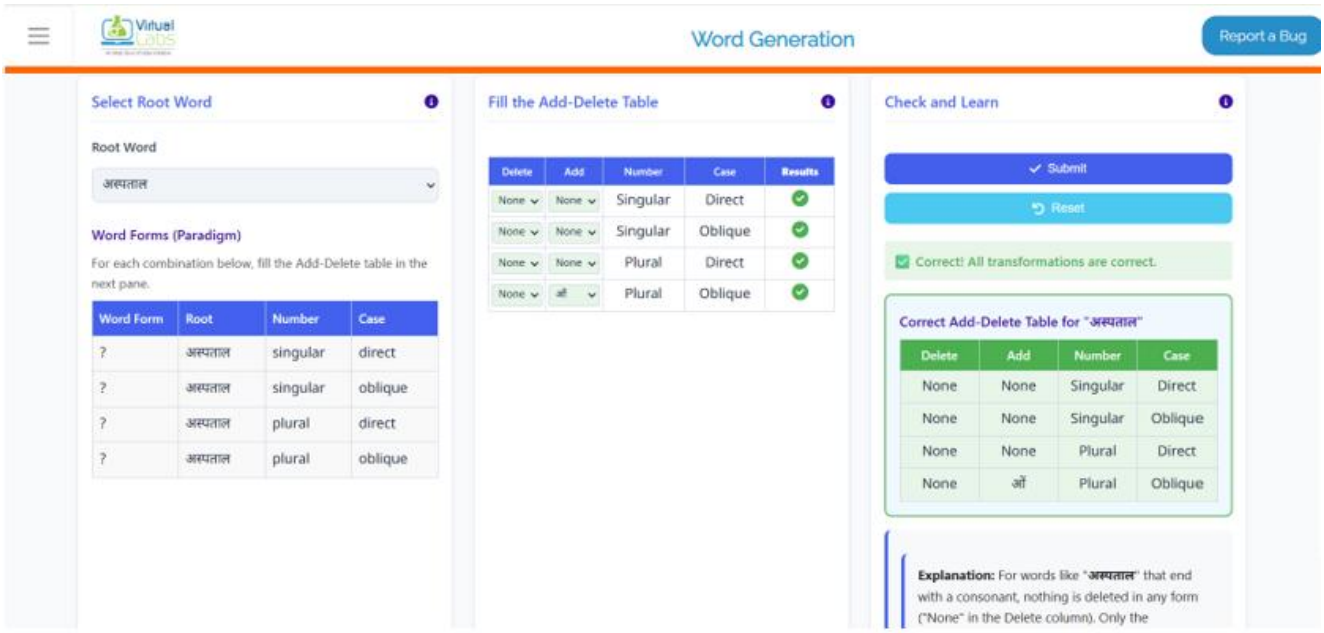
Algorithm:

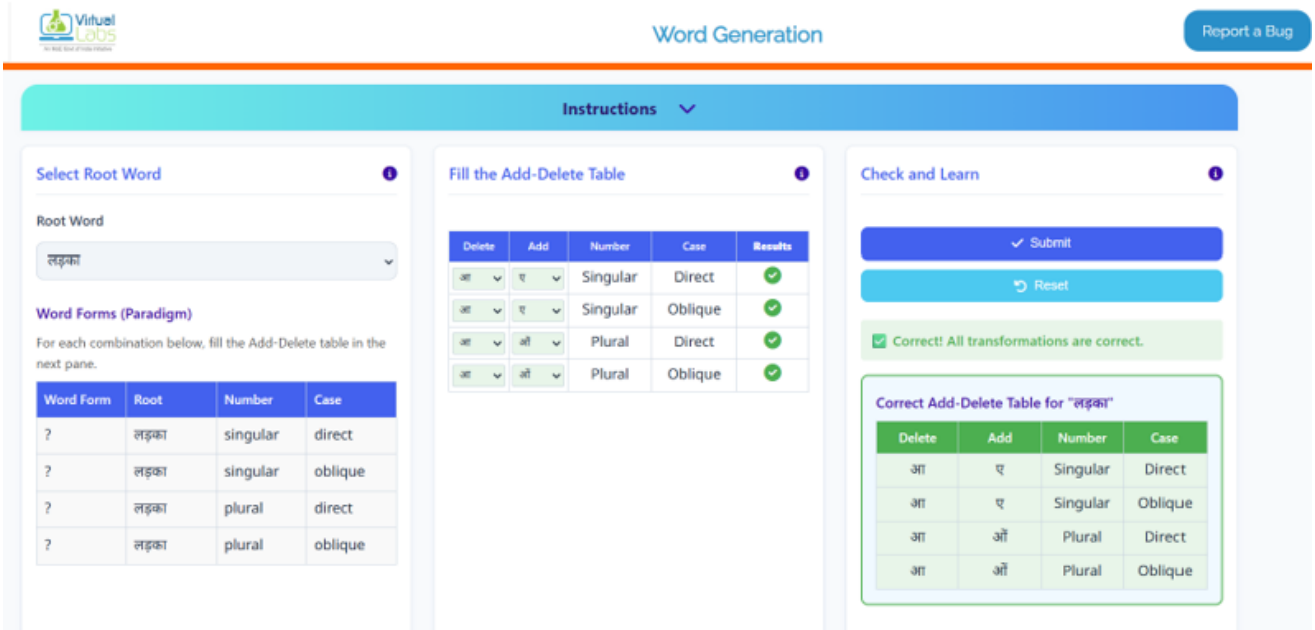
1. Start the program
2. Input root word and features
3. Identify category (noun/verb/adjective)
4. Apply morphological rules:
    - Hindi suffix rules (आ → ए, ओं, इय ◌ाँ, etc.)
    - English suffix rules (-s, -ed, -ing)
5. Generate word form
6. Display result

Procedure:

1. Input training text
2. Preprocess text (tokenization, cleaning)
3. Generate N-grams from the text
4. Calculate probability distribution of words
5. Select initial word (seed)
6. Predict next word based on probabilities
7. Append predicted word to sequence
8. Repeat until desired length is achieved
9. Output generated word sequence

Output:


Virtual Labs
Word Generation
Report a Bug
Select Root Word
Root Word
अस्पताल
Word Forms (Paradigm)
For each combination below, fill the Add-Delete table in the next pane.
Word Form | Root | Number | Case
? | अस्पताल | singular | direct
? | अस्पताल | singular | oblique
? | अस्पताल | plural | direct
? | अस्पताल | plural | oblique
Fill the Add-Delete Table
Delete | Add | Number | Case | Results
None | None | Singular | Direct
None | None | Singular | Oblique
None | None | Plural | Direct
None | ओं | Plural | Oblique
Check and Learn
Submit
Reset
Correct! All transformations are correct.
Correct Add-Delete Table for "अस्पताल"
Delete | Add | Number | Case
None | None | Singular | Direct
None | None | Singular | Oblique
None | None | Plural | Direct
None | ओं | Plural | Oblique
Explanation: For words like "अस्पताल" that end with a consonant, nothing is deleted in any form ("None" in the Delete column). Only the


Virtual Labs
Word Generation
Report a Bug
Instructions
Select Root Word
Root Word
लड़का
Word Forms (Paradigm)
For each combination below, fill the Add-Delete table in the next pane.
Word Form | Root | Number | Case
? | लड़का | singular | direct
? | लड़का | singular | oblique
? | लड़का | plural | direct
? | लड़का | plural | oblique
Fill the Add-Delete Table
Delete | Add | Number | Case | Results
आ | ए | Singular | Direct
आ | ए | Singular | Oblique
आ | ओं | Plural | Direct
आ | ओं | Plural | Oblique
Check and Learn
Submit
Reset
Correct! All transformations are correct.
Correct Add-Delete Table for "लड़का"
Delete | Add | Number | Case
आ | ए | Singular | Direct
आ | ए | Singular | Oblique
आ | ओं | Plural | Direct
आ | ओं | Plural | Oblique

Result:

This experiment demonstrates how word generation depends on grammatical features like gender, number, tense, and case.

Hindi shows more variation due to gender and case marking, while English relies more on suffixes and fixed structures. Understanding these rules helps in natural language processing and grammar modeling.

References:

1.https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/

2.https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/

3. https://www.tutorialspoint.com/natural_language_processing/index.htm